Tecnológico de Monterrey

Escuela de Ingeniería y Ciencias

Diseño de Redes Neuronales y Aprendizaje Profundo (TC2035.201)

Clasificación automática de 104 tipos de flores con redes neuronales profundas

Situación problema: Selección de datos

Petals to the Metal – Flower Classification on TPU (Kaggle)

Equipo

Nombre	Matrícula
Angel Eduardo Luna Lejia	A00842657
Andrea Estefanía De la Peña Contreras	A00840973
Jannette Cristina Flores Rodríguez	A01255711
Rey Iván de la Fuente Chávez	A01709445
Andrés Huerta Robinson	A00838626

Monterrey, Nuevo León

25 de octubre de 2026 (versión corregida)

Índice

1. Contexto del problema **3**

2. Descripción del dataset **3**

2.1. Origen . . . 3
2.2. Tamaño y estructura . . . 4
2.3. Ejemplos de las imágenes . . . 4
2.4. Balance de clases y métrica de evaluación . . . 5
2.5. Implicaciones para el proyecto . . . 7

3. Algoritmos y aplicaciones **7**

3.1. Por qué redes neuronales convolucionales . . . 7
3.2. Propuesta 1: CNN propia como línea base . . . 8
3.3. Propuesta 2: aprendizaje por transferencia con EfficientNetB0 o ResNet50 . . . 8
3.4. Métodos tradicionales: solo sobre características . . . 9
3.5. Otras arquitecturas consideradas . . . 10
3.6. Aplicaciones posibles . . . 10

Resumen

Este trabajo aborda el problema de clasificación automática de imágenes de flores utilizando redes neuronales profundas. Se utiliza el conjunto de datos de la competencia *Petals to the Metal – Flower Classification on TPU* de Kaggle, compuesto por 23,847 imágenes RGB distribuidas entre 104 clases distintas de flores. El conjunto presenta variaciones importantes en tamaño, color, iluminación, orientación y composición de las imágenes, además de un desbalance marcado: en entrenamiento, la clase más pequeña tiene 18 imágenes y la más grande 782, con una mediana de 88.

La tarea consiste en desarrollar un modelo capaz de identificar correctamente la clase de una flor a partir de una imagen. Debido a la similitud visual entre algunas categorías y a la variabilidad dentro de una misma clase, se proponen dos arquitecturas concretas: una red neuronal convolucional (CNN) propia, entrenada desde cero como línea base, y una red preentrenada en ImageNet (EfficientNetB0, con ResNet50 como alternativa) adaptada mediante aprendizaje por transferencia, que es la opción más adecuada para el número reducido de imágenes por clase. Los métodos tradicionales (KNN, bosques aleatorios y SVM) se analizan como referencia y se explica por qué solo son viables sobre características extraídas y no sobre los 150,528 valores de cada imagen.

Para evaluar el desempeño se utiliza la métrica F1 macro de Kaggle, ya que asigna la misma importancia a cada una de las 104 clases independientemente de la cantidad de imágenes disponibles. De esta forma, el proyecto busca una estrategia de clasificación que logre un buen desempeño general sin favorecer únicamente a las clases con mayor cantidad de ejemplos.

1. Contexto del problema

Saber qué planta es la que aparece en una fotografía es el primer paso de muchas actividades: registrar biodiversidad, vigilar especies en riesgo, enseñar botánica o controlar la calidad de un producto agrícola. Hoy esa identificación depende, en gran medida, de que una persona con conocimiento especializado revise cada imagen, y ese recurso no alcanza para la cantidad de fotografías que se generan.

El caso más claro es la **ciencia ciudadana**. La plataforma iNaturalist ha acumulado más de 300 millones de observaciones subidas por personas comunes (iNaturalist, 2026); si cada una tuviera que ser identificada a mano por un experto, la mayoría quedaría sin clasificar. Esto es relevante para la conservación, ya que el informe *State of the World's Plants and Fungi* estima que el 39.4 % de las especies de plantas está amenazado de extinción (Antonelli et al., 2020), y no se puede monitorear una especie si no se sabe dónde aparece. Los **jardines botánicos** y los museos enfrentan el mismo problema a menor escala: tienen colecciones y visitantes que preguntan qué están viendo, pero poco personal para responder.

También existe una necesidad económica. En México se cultivaron alrededor de 46 tipos de flores en 2021, con un valor de producción de 9,590 millones de pesos; tan solo el clavel aportó 839 millones, y el Estado de México generó el 81.4 % del valor de las flores de temporada otoñal (Servicio de Información Agroalimentaria y Pesquera, 2022). En el **comercio de flores**, distinguir variedades de forma automática ayuda a clasificar producto, verificar envíos y llevar inventarios sin depender de inspección visual.

El problema de fondo es que la diversidad es enorme: se estima que hay más de 5,000 especies de mamíferos, 10,000 de aves y 30,000 de peces, pero más de 400,000 tipos de flores (Kaggle, 2020). Muchas de ellas son visualmente muy parecidas, por lo que distinguirlas requiere experiencia, e identificarlas manualmente es lento y no escala cuando se tienen miles de fotografías. Por eso resulta valioso contar con modelos que reconozcan el tipo de flor de forma automática a partir de una imagen.

Este proyecto parte de la competencia *Petals to the Metal*, de Kaggle, cuyo reto es construir un modelo que identifique el tipo de flor en un conjunto de imágenes. La competencia se limita a 104 clases de flores y corresponde a un problema de clasificación multiclase de imágenes en el que el modelo debe captar detalles finos de forma, estructura y color para distinguir entre categorías cercanas.

El problema tiene varias dificultades que lo hacen relevante desde un enfoque de redes neuronales. Primero, las clases son heterogéneas: algunas contienen un solo subtipo de flor y otras agrupan muchas subvariedades. Segundo, las imágenes no se capturaron en el mismo entorno, lo que agrega variación de fondo, escala e iluminación. Por último, el número de imágenes por clase no es uniforme (sección 2.4), por lo que, dada la métrica de evaluación, el modelo debe funcionar bien tanto en las clases con pocos ejemplos como en las que tienen muchos.

2. Descripción del dataset

2.1. Origen

Los datos vienen de *Petals to the Metal – Flower Classification on TPU*, una competencia *Getting Started* de Kaggle cuyo objetivo es aprender a entrenar modelos en TPU (Kaggle, 2020). Según la descripción de la competencia, las imágenes se tomaron de cinco conjuntos de datos públicos. Algunas clases son muy específicas (por ejemplo, *pink primrose*, que contiene un solo subtipo de flor) y otras agrupan muchas subvariedades (por ejemplo, *wild rose*).

Al revisar la lista de clases nos dimos cuenta de que las primeras 102 siguen exactamente el orden del conjunto Oxford 102 Flowers (Nilsback & Zisserman, 2008): 94 conservan el mismo

nombre y las otras 8 se renombraron, se corrigieron o se ampliaron (por ejemplo, *oxeye daisy* pasó a ser *daisy* y *bearded iris* pasó a ser *iris*). Las dos clases extra son *common tulip* y *wild rose*. Las etiquetas están en inglés, así que usamos los nombres originales en cursiva.

2.2. Tamaño y estructura

En total hay 23,847 imágenes a color (RGB) de 104 clases, repartidas como muestra la Tabla 1. Vienen en formato TFRecord, que junta muchas imágenes en pocos archivos binarios para que la TPU las lea rápido. Cada registro tiene tres campos: `image` (la foto codificada en JPEG), `id` (un identificador único) y `class` (la etiqueta, un entero de 0 a 103). Los registros de prueba no traen `class`, porque esas etiquetas solo las tiene Kaggle para calificar.

Tabla 1: Particiones del conjunto de datos.

Partición	Imágenes	%	¿Etiquetado?
Entrenamiento	12,753	53.5	Sí
Validación	3,712	15.6	Sí
Prueba	7,382	31.0	No
Total	23,847	100.0	

Cada partición viene en 4 resoluciones: 192×192, 224×224, 331×331 y 512×512 píxeles, con 16 archivos TFRecord por partición, por lo que la base de datos trae 192 archivos TFRecord (4 resoluciones $\times$ 3 particiones $\times$ 16 archivos) más `sample_submission.csv`. Tener varias resoluciones nos conviene porque no todas las arquitecturas se diseñaron para el mismo tamaño de entrada: ResNet50 y EfficientNetB0 usan 224×224, mientras que las EfficientNet más grandes usan resoluciones mayores. Para dimensionar el problema, una sola imagen de 224×224 ya tiene $224 \times 224 \times 3 = 150{,}528$ valores.

2.3. Ejemplos de las imágenes

La Tabla 2 muestra algunas clases concretas y lo que cada una nos dice sobre el problema. Además, dentro de una misma clase, las fotos cambian mucho de escala, pose e iluminación (Nilsback & Zisserman, 2008). La Figura 1 muestra fotos reales del conjunto de entrenamiento.



Figura 1: Ejemplos reales del conjunto de entrenamiento con su clase y número de etiqueta.

Tabla 2: Clases de ejemplo (nombre y etiqueta) y lo que ilustran.

Clase (etiqueta)	Qué ilustra
sunflower (53)	Girasol. Flor grande y muy característica. Posiblemente sea de las más fáciles de identificar.
daisy (48), *barberton daisy* (40), *osteospermum* (65), *black-eyed susan* (62)	Margarita, gerbera, osteospermum y rudbeckia. Todas tienen pétalos alrededor de un disco central. Pueden surgir confusiones entre ellas.
sword lily (42), *tiger lily* (5), *fire lily* (20), *peruvian lily* (17)	Gladiola, lirio tigre, gloriosa y alstroemeria. Todas dicen *lily*, pero son de cuatro familias botánicas distintas: el modelo tiene que fijarse en la forma, no en el nombre.
iris (67), *yellow iris* (14)	Misma forma general, la diferencia principal es el color.
rose (73), *wild rose* (103)	Dos clases de rosas. *Wild rose* es amplia (varias subvariedades) y no viene de Oxford 102.
common tulip (102)	Tulipán. Clase amplia que tampoco viene de Oxford 102.
pink primrose (0)	Clase muy específica con un solo subtipo de flor.
carnation (30)	Clavel, una de las flores comerciales con mayor valor de producción en México (Servicio de Información Agroalimentaria y Pesquera, 2022).

2.4. Balance de clases y métrica de evaluación

Con 12,753 imágenes de entrenamiento y 104 clases, tenemos en promedio 122.6 imágenes por clase. Sin embargo, las clases no tienen el mismo tamaño: como unas agrupan muchas subvariedades y otras solo una, algunas tienen muchas más imágenes que otras. Contamos las imágenes de cada clase en los TFRecord de entrenamiento y validación; la Tabla 3 resume el resultado y la Figura 2 muestra la distribución completa.

Tabla 3: Número de imágenes de entrenamiento por clase.

Estadístico	Imágenes
Mínimo (*moon orchid*, *alpine sea holly*, *bolero deep blue*)	18
Primer cuartil	32.5
Mediana	88
Media	122.6
Tercer cuartil	125.5
Máximo (*iris*)	782
Razón máximo/mínimo	43:1
Clases con menos de 30 imágenes	24 de 104
Clases por debajo de la media	76 de 104
Imágenes en las 10 clases más grandes	40 %

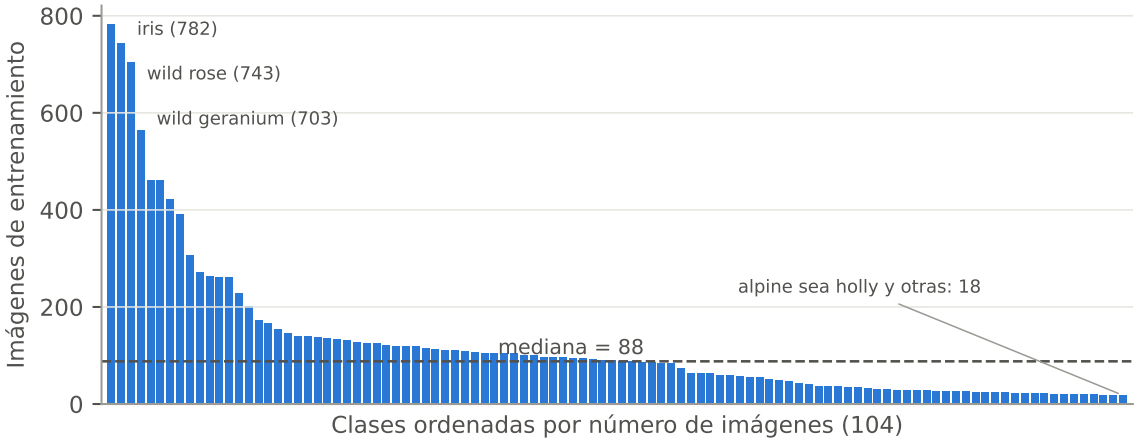


Figura 2: Imágenes de entrenamiento por clase, ordenadas de mayor a menor. La línea punteada marca la mediana (88).

La media (122.6) queda muy por encima de la mediana (88) porque unas pocas clases concentran los datos: las 10 más grandes reúnen el 40 % de las imágenes, mientras que la mitad de las clases tiene 88 imágenes o menos. Validación conserva las mismas proporciones, con entre 5 y 228 imágenes por clase, así que las clases raras también tienen muy pocos ejemplos para evaluarlas: en una clase con 5 imágenes de validación, un solo acierto más o menos cambia mucho su F1.

Por este desbalance, la exactitud no basta: un modelo puede acertar mucho en las clases grandes, fallar en las clases pequeñas y aún verse bien. Por esto mismo, Kaggle califica con el F1 macro (Kaggle, 2020). Para cada clase c se calcula la precisión P_c, la sensibilidad (*recall*) R_c y su media armónica:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \tag{1}$$

$$F1_c = \frac{2\,P_c\,R_c}{P_c + R_c}, \qquad F1_{\text{macro}} = \frac{1}{104}\sum_{c=0}^{103} F1_c, \tag{2}$$

donde TP_c, FP_c y FN_c son los verdaderos positivos, falsos positivos y falsos negativos de la clase c. Así, todas las clases pesan lo mismo, sin importar cuántas imágenes tengan.

2.5. Implicaciones para el proyecto

Como Kaggle ya separa entrenamiento y validación, no necesitamos hacer nuestra propia partición: entrenaremos con `train`, elegiremos modelos e hiperparámetros con `val` y usaremos `test` solo para la predicción final. Las imágenes vienen en JPEG con valores de 0 a 255, así que habrá que decodificarlas y escalarlas (a $[0, 1]$ o con el preprocesamiento que pida cada red preentrenada). Como las etiquetas son enteros, la red producirá 104 valores de salida, uno por cada clase, que se transformarán en probabilidades mediante Softmax, y se entrenará con entropía cruzada categórica dispersa. Como las clases están desbalanceadas, siempre reportaremos el F1 macro y no solo la exactitud, y revisaremos por separado el desempeño en las clases pequeñas.

3. Algoritmos y aplicaciones

El problema es de visión por computadora: la entrada es una imagen de $224 \times 224 \times 3 = 150{,}528$ valores y la salida es una de 104 clases. En esta sección explicamos por qué las redes convolucionales son la familia adecuada, proponemos dos arquitecturas concretas, explicamos qué papel pueden tener los métodos tradicionales y describimos las aplicaciones posibles.

3.1. Por qué redes neuronales convolucionales

Las redes neuronales convolucionales (CNN) aplican filtros pequeños (por ejemplo, de 3×3) que recorren toda la imagen. En las primeras capas estos filtros aprenden a detectar bordes y cambios de color; en las capas profundas, combinaciones de ellos, como texturas, pétalos o discos centrales (Figura 3). En las capas internas se usa la activación ReLU y en la salida Softmax, que convierte los 104 valores finales en probabilidades.

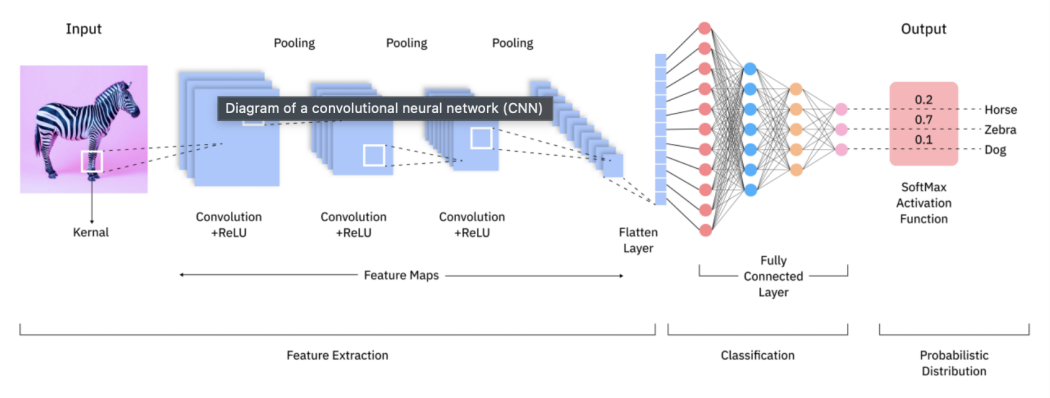


Figura 3: Una CNN descompone la imagen en mapas de características y después las combina para clasificar (IBM, s.f.).

Para este conjunto de datos, la CNN tiene dos ventajas concretas sobre una red densa:

- **Comparte parámetros.** Una capa convolucional de 32 filtros de 3×3 sobre una imagen RGB tiene $3 \cdot 3 \cdot 3 \cdot 32 + 32 = 896$ parámetros, sin importar el tamaño de la imagen. Una capa densa de 256 neuronas conectada directamente a los 150,528 valores tendría $150{,}528 \cdot 256 + 256 \approx 38.5$ millones, más de 3,000 parámetros por cada imagen de entrenamiento.

- **Reconoce un patrón en cualquier posición.** El mismo filtro se aplica en toda la imagen, así que una flor se reconoce aunque aparezca en una esquina o en el centro, algo indispensable porque nuestras fotos cambian mucho de escala y encuadre.

La desventaja es el costo de cómputo, ya que entrenar una CNN requiere muchas operaciones por imagen. Por eso la competencia ofrece TPU y por eso también buscamos arquitecturas eficientes.

3.2. Propuesta 1: CNN propia como línea base

La primera arquitectura es una CNN diseñada por nosotros y entrenada desde cero. Su objetivo es medir cuánto se puede aprender usando únicamente las 12,753 imágenes de entrenamiento y servir de referencia para la segunda propuesta. La arquitectura es:

1. Cuatro bloques convolucionales con 32, 64, 128 y 256 filtros. Cada bloque tiene una convolución de 3×3, normalización por lotes (*batch normalization*), ReLU y *max pooling* de 2×2, que reduce a la mitad el alto y el ancho. Así la red pasa de 224×224 a mapas de 14×14 con 256 canales.
2. *Global Average Pooling*, que resume cada uno de los 256 mapas en un solo número.
3. Una capa densa de 256 neuronas con ReLU y *Dropout* de 0.5 para reducir el sobreajuste.
4. Una capa de salida de 104 neuronas con Softmax.

En total son aproximadamente 0.48 millones de parámetros, una cantidad pequeña a propósito. Con una mediana de 88 imágenes por clase y 24 clases con menos de 30, una red grande entrenada desde cero memorizaría el conjunto de entrenamiento. Para compensar la falta de datos usaremos aumento de datos (volteos, rotaciones pequeñas, recortes y cambios de brillo) solo en entrenamiento.

Esperamos que esta red aprenda bien las clases grandes y fáciles (como *sunflower*), pero que tenga problemas con las clases pequeñas y con las que se parecen entre sí (Tabla 2), porque necesita aprender desde cero incluso los bordes y texturas más básicos.

3.3. Propuesta 2: aprendizaje por transferencia con EfficientNetB0 o ResNet50

La segunda arquitectura reutiliza una red ya entrenada en ImageNet, un conjunto de 1.28 millones de imágenes de 1,000 categorías (Russakovsky et al., 2015). Esta es la propuesta principal, por tres razones:

- **Las primeras capas sirven para cualquier imagen.** Los filtros iniciales de una red entrenada en ImageNet detectan bordes, colores y texturas que son útiles en otras tareas, y transferirlos mejora el resultado aun cuando la tarea nueva es distinta (Yosinski et al., 2014). Con aproximadamente 123 imágenes por clase no podríamos aprender esos filtros nosotros mismos; al heredarlos, solo hay que aprender a combinarlos para distinguir 104 flores.
- **Ya se probó con flores.** En Oxford 102, de donde vienen 102 de nuestras 104 clases, usar las características de una CNN entrenada en ImageNet con un clasificador lineal alcanzó 86.8 % de exactitud promedio (Razavian et al., 2014), frente a 72.8 % del método original con características diseñadas a mano (Nilsback & Zisserman, 2008).

- **Pocos parámetros nuevos.** Se retira la capa de salida de ImageNet y se agrega una nueva de 104 clases. Sobre las 1,280 características de EfficientNetB0, esta capa tiene $1{,}280 \cdot 104 + 104 = 133{,}224$ parámetros, que sí se pueden ajustar bien con 12,753 imágenes.

Elegimos **EfficientNetB0** como primera opción y **ResNet50** como alternativa (Tabla 4). EfficientNetB0 escala de forma equilibrada la profundidad, el ancho y la resolución de la red, y por eso obtiene mayor exactitud en ImageNet con 5 veces menos parámetros que ResNet50 (Tan & Le, 2019). ResNet50 introduce conexiones residuales, que suman la entrada de cada bloque a su salida y permiten entrenar redes profundas sin que el gradiente se desvanezca (He et al., 2016); es más pesada, pero es una arquitectura muy estudiada y sirve para confirmar que los resultados no dependen de una sola red. Ambas usan entradas de 224×224, que es una de las resoluciones que trae el conjunto de datos.

Tabla 4: Redes preentrenadas consideradas (Tan & Le, 2019).

Red	Parámetros (millones)	Entrada (píxeles)	Exactitud top-1 ImageNet (%)	Características tras el *pooling*
EfficientNetB0	5.3	224×224	77.1	1,280
ResNet50	26	224×224	76.0	2,048

El entrenamiento se hará en dos fases:

1. **Extracción de características.** Se congelan los pesos de la red preentrenada y solo se entrena la nueva capa de salida (con *Dropout* antes de ella). Así se evita que los gradientes grandes del inicio dañen los filtros aprendidos en ImageNet.

2. **Ajuste fino** (*fine-tuning*). Se descongelan los últimos bloques y se entrena toda la parte superior con una tasa de aprendizaje baja (del orden de 10^{-5}), para adaptar las características de alto nivel a las flores sin olvidar las de bajo nivel.

En ambas fases se usará el mismo aumento de datos que en la propuesta 1 y el preprocesamiento que pide cada red. Además, para atender el desbalance, compararemos la entropía cruzada normal contra una ponderada por clase, y elegiremos el modelo por su F1 macro en validación.

3.4. Métodos tradicionales: solo sobre características

Los métodos clásicos de aprendizaje automático no son adecuados si se aplican directamente sobre los píxeles, porque cada imagen sería un vector de 150,528 dimensiones y solo tenemos 12,753 ejemplos, es decir, más dimensiones que ejemplos:

k vecinos más cercanos (KNN) Clasifica una imagen según la clase de las k imágenes más cercanas, normalmente con distancia euclidiana. Sobre píxeles, esa distancia mide si dos fotos tienen colores parecidos en las mismas posiciones, no si muestran la misma flor: dos fotos de girasoles con distinto encuadre quedan lejos, y un girasol y una rudbeckia sobre fondo verde quedan cerca. Además, en dimensiones tan altas las distancias entre todos los puntos tienden a parecerse.

Bosque aleatorio Combina muchos árboles de decisión, cada uno entrenado con una muestra distinta de los datos, y clasifica por votación. Cada división del árbol pregunta por el valor de un píxel en una posición fija, pero la flor no aparece siempre en la misma posición, así que esas reglas no generalizan.

Máquinas de vectores de soporte (SVM) Buscan el hiperplano que separa dos clases con el mayor margen; para 104 clases se entrena un clasificador por clase (uno contra todos). Sobre píxeles, un SVM lineal tendría $104 \cdot 150{,}529 \approx 15.7$ millones de pesos, y seguiría sin ser robusto a cambios de posición o iluminación.

Estos métodos sí son útiles cuando reciben un vector de características compacto en lugar de los píxeles. Así trabajó el método original de Oxford 102, que usó un SVM sobre descriptores de color, forma (SIFT) y gradientes (HOG) (Nilsback & Zisserman, 2008). Del mismo modo, Razavian et al. obtuvieron su resultado de 86.8 % con un SVM lineal sobre las características de una CNN preentrenada (Razavian et al., 2014). Por eso los usaremos como comparación sobre los vectores de 1,280 características que produce EfficientNetB0: si un KNN o un SVM sobre esas características se acerca a la red con ajuste fino, sabremos cuánto aporta realmente el ajuste fino.

3.5. Otras arquitecturas consideradas

AlexNet fue la primera CNN profunda en ganar ImageNet, en 2012 (Krizhevsky et al., 2012), e Inception (GoogLeNet) introdujo bloques que aplican filtros de varios tamaños en paralelo (Szegedy et al., 2015). Ambas son importantes históricamente, pero no las elegimos: AlexNet tiene unos 60 millones de parámetros y menor exactitud que las redes de la Tabla 4, y las versiones de Inception que superan a EfficientNetB0 requieren más parámetros y una entrada de 299×299. Si EfficientNetB0 resultara insuficiente, el siguiente paso sería una EfficientNet más grande con las imágenes de 331×331 o 512×512.

3.6. Aplicaciones posibles

Un modelo que clasifica flores a partir de imágenes responde directamente a las necesidades descritas en el contexto. La primera es la **identificación automática de plantas** en plataformas de ciencia ciudadana: iNaturalist, con más de 300 millones de observaciones, usa un modelo de visión por computadora entrenado con esas mismas observaciones para sugerir la especie a partir de una fotografía (iNaturalist, 2026). Sin ese apoyo, cada observación dependería de que una persona la revisara, y el monitoreo de biodiversidad sería lento y difícil de escalar. Este proyecto es una versión a menor escala de ese tipo de sistemas.

La segunda es una **herramienta educativa** para estudiantes o visitantes de jardines botánicos, que permita fotografiar una flor y consultar información sobre ella. La tercera es el **apoyo al comercio de flores**, por ejemplo para clasificar o verificar variedades comerciales como el clavel, que forma parte de las 104 clases.

Sin embargo, el modelo desarrollado solo podrá reconocer las 104 clases presentes en el conjunto de datos, por lo que una aplicación real más amplia requeriría entrenarlo con una mayor variedad de especies y con fotografías tomadas en las condiciones de uso reales.

Referencias

Antonelli, A., Fry, C., Smith, R. J., Simmonds, M. S. J., Kersey, P. J., Pritchard, H. W., et al. (2020). *State of the World's Plants and Fungi 2020*. Royal Botanic Gardens, Kew. https://doi.org/10.34885/172

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition*, 770-778. https://doi.org/10.1109/CVPR.2016.90

IBM. (s.f.). *What is image classification?* Consultado el 28 de septiembre de 2026, desde https://www.ibm.com/think/topics/image-classification

iNaturalist. (2026). *iNaturalist: A community for naturalists*. Consultado el 28 de septiembre de 2026, desde https://www.inaturalist.org

Kaggle. (2020). *Petals to the Metal: Flower Classification on TPU*. Consultado el 28 de septiembre de 2026, desde https://www.kaggle.com/competitions/tpu-getting-started/overview

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, *25*, 1097-1105. Consultado el 25 de octubre de 2026, desde https://papers.nips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html

Nilsback, M.-E., & Zisserman, A. (2008). Automated flower classification over a large number of classes. *2008 Sixth Indian Conference on Computer Vision, Graphics & Image Processing*, 722-729. https://doi.org/10.1109/ICVGIP.2008.47

Razavian, A. S., Azizpour, H., Sullivan, J., & Carlsson, S. (2014). CNN features off-the-shelf: An astounding baseline for recognition. *2014 IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 512-519. https://doi.org/10.1109/CVPRW.2014.131

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., & Fei-Fei, L. (2015). ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, *115*(3), 211-252. https://doi.org/10.1007/s11263-015-0816-y

Servicio de Información Agroalimentaria y Pesquera. (2022). *Floricultura mexicana: aporte en la celebración del Día de Muertos*. Secretaría de Agricultura y Desarrollo Rural. Consultado el 25 de octubre de 2026, desde https://www.gob.mx/cms/uploads/attachment/file/769150/BOLETI_N_FLORICULTURA_DI_A_DE_MUERTOS_22.pdf

Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., & Rabinovich, A. (2015). Going deeper with convolutions. *2015 IEEE Conference on Computer Vision and Pattern Recognition*, 1-9. https://doi.org/10.1109/CVPR.2015.7298594

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *Proceedings of the 36th International Conference on Machine Learning*, *97*, 6105-6114. Consultado el 25 de octubre de 2026, desde https://proceedings.mlr.press/v97/tan19a.html

Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks? *Advances in Neural Information Processing Systems*, *27*, 3320-3328. Consultado el 25 de octubre de 2026, desde https://proceedings.neurips.cc/paper/2014/hash/375c71349b295fbe2dcdca9206f20a06-Abstract.html